任务 1：数据库与查询条件基础说明（3 分）

1.页面中的输入为什么会与数据库查询有关

答：页面输入与数据库查询的关联原因：DVWA 查询用户信息的页面，核心业务功能是根据用户输入的内容筛选数据，网站的用户账号、信息记录等所有数据都统一存储在后端数据库中。页面本身无法独立存储、读取大量数据，必须调用数据库完成数据调取，因此前端页面的用户输入，会直接对接后端数据库查询逻辑。

2.一个最小业务链路是什么

输入条件 → 代码拼接 SQL 语句 → 数据库查询解析执行 → 数据筛选匹配 → 页面返回结果

3.用户输入是如何进入查询条件的

答：在 DVWA 低安全等级配置下，后端代码无过滤、无分离处理：

浏览器前端输入框中，用户自主填写字符、数字等查询内容；

后端程序直接获取前端原始输入数据；

将用户输入直接以字符串形式拼接到原生 SQL 查询语句的条件位中；

拼接完成的完整 SQL 语句，直接提交给数据库执行。

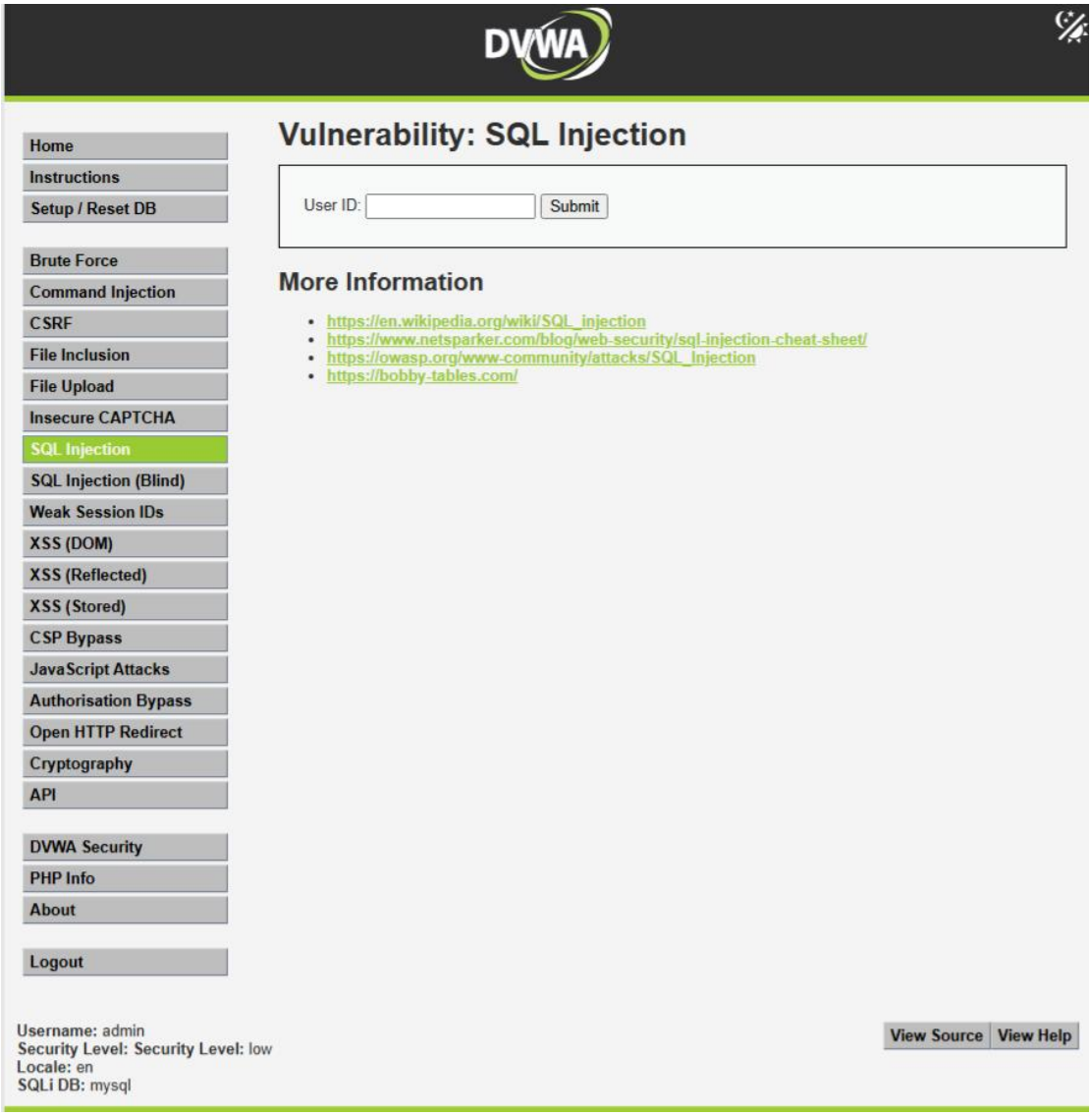
4.为什么这为 SQL 注入创造了可能性

正常业务中，用户输入仅应作为查询参数值，只负责匹配数据；但此处采用字符串直接拼接的写法，没有对输入内容做限制。用户可以刻意输入含 SQL 语法的特殊字符、指令，这些恶意字符会破坏原有 SQL 语句结构，篡改查询逻辑，让数据库执行开发者意料之外的命令，由此直接创造了 SQL 注入的利用条件。

任务 2：DVWA SQL Injection Low 现象记录（4 分）

1.实验页面或实验目标：

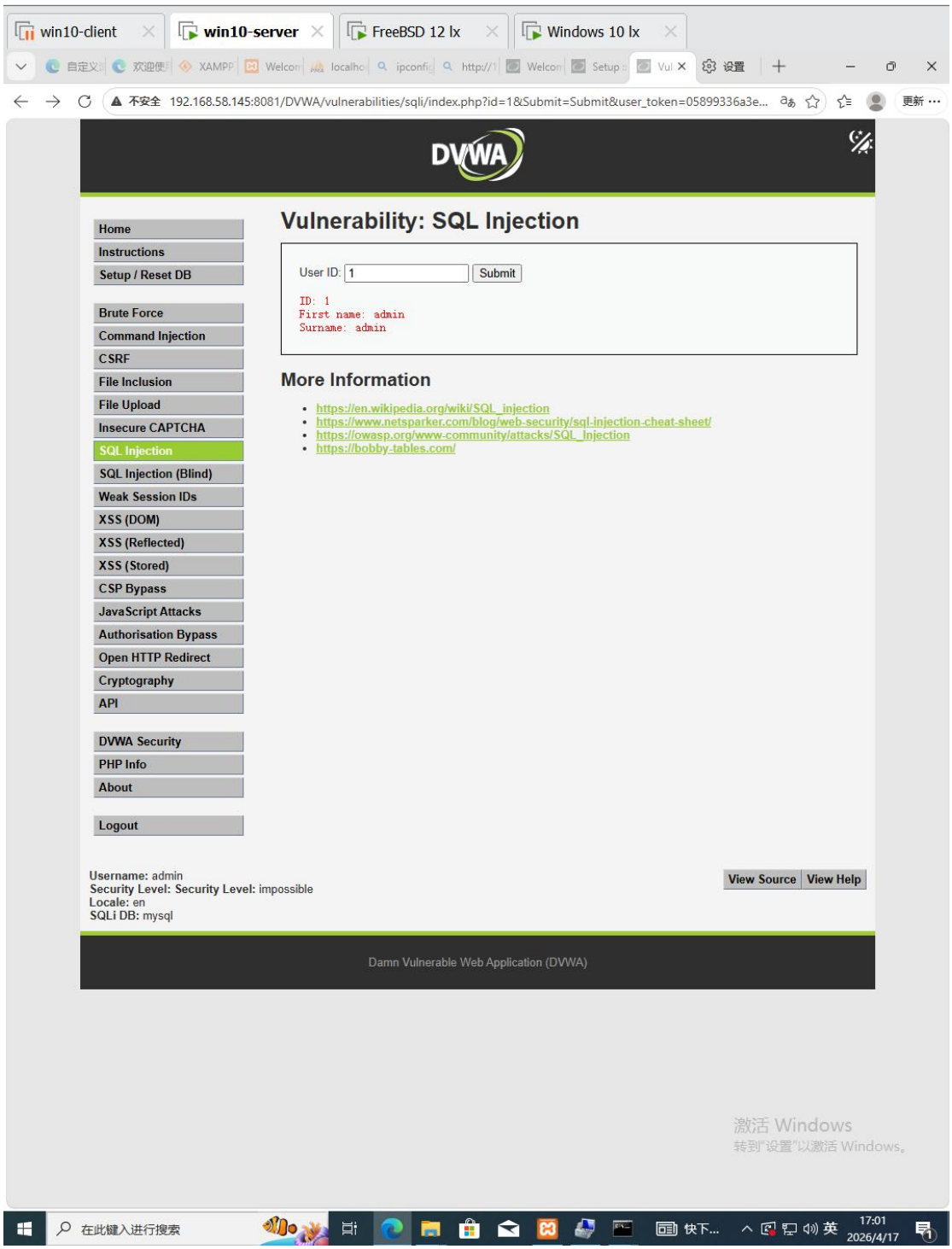
实验页面：



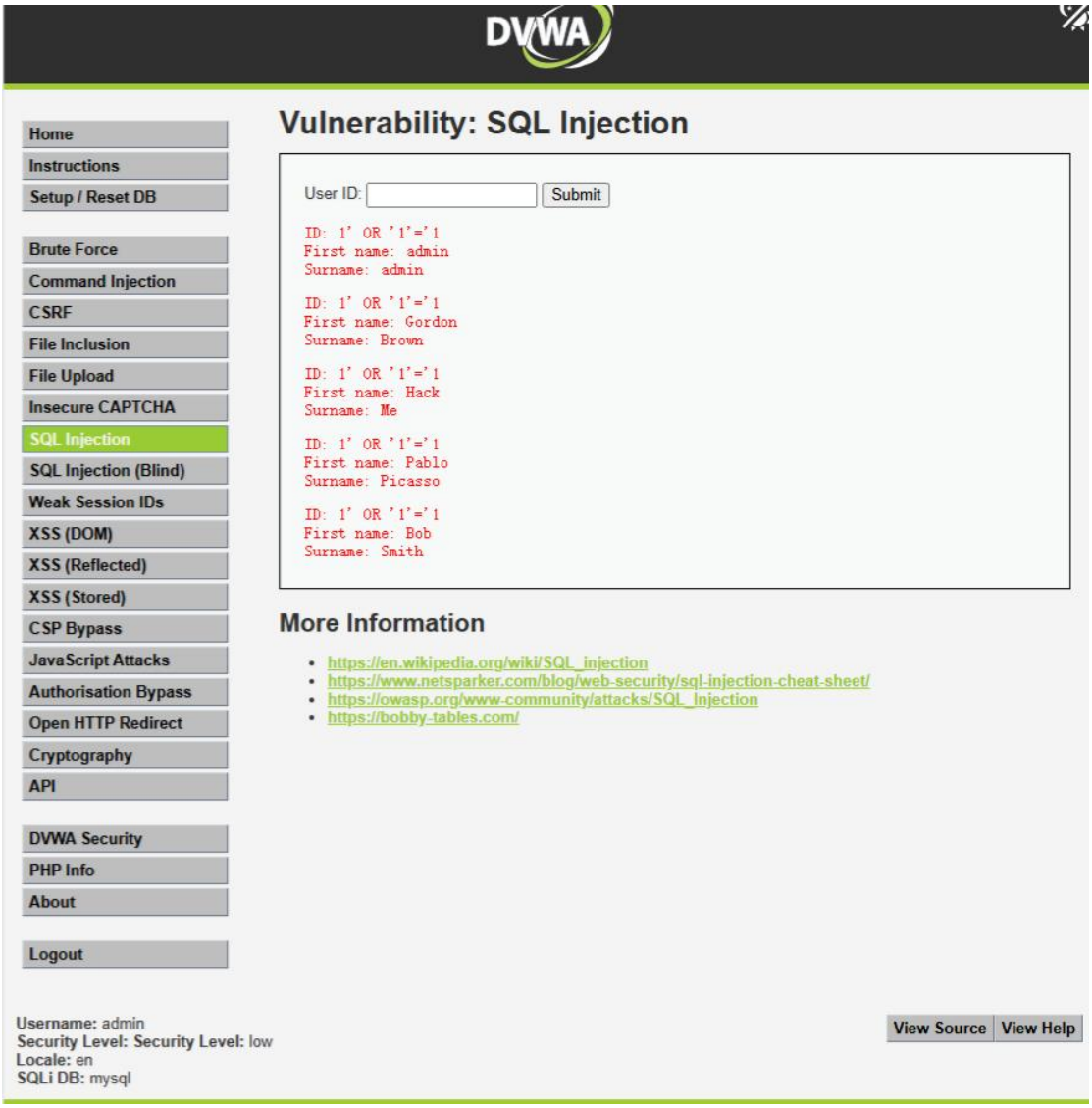
实验目标：在教师授权的 DVWA 环境（虚拟机访问 localhost/DVWA），完成 SQL Injection 模块（Low 安全等级）的基础现象观察与记录；

2. 普通输入与异常输入的对比：

普通输入：


win10-client
win10-server
FreeBSD 12 lx
Windows 10 lx
192.168.58.145:8081/DVWA/vulnerabilities/sqli/index.php?id=1&Submit=Submit&user_token=05899336a3e...
DVWA
Vulnerability: SQL Injection
User ID: 1 Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
Username: admin
Security Level: Security Level: impossible
Locale: en
SQLi DB: mysql
View Source
View Help
Damn Vulnerable Web Application (DVWA)
激活 Windows
转到"设置"以激活 Windows。
在此键入进行搜索
17:01
2026/4/17

异常输入：



普通输入与异常输入对比解释和现象说明：

输入类型	具体输入内容	现象说明（截图辅助，截图脱敏处理）
普通输入	输入“1”（纯数字，对应数据库中存在的用户 ID）	【截图说明】截图可清晰看到“1”的输入内容，页面无报错； 【支撑判断】该输入符合页面“查询 ID”的业务逻辑，属于正常用户操作，支撑“合法输入可触发正常查询”的判断。
异常输入	输入“1’ OR ’1’=’1”（含 SQL 特殊字符，恶意拼接逻辑）	【截图说明】截图可清晰看到特殊字符，页面无明显报错，返回结果与普通输入差异显著； 【支撑判断】该输入超出“单纯输入 ID”的业务预期，携带 SQL 语法字符，支撑“异常输入可篡改查询逻辑”的判断。

3. 返回结果的差异

输入类型	具体返回结果	差异说明及支撑判断
普通输入	输入“1”：仅返回 ID=1 的用户信息，无多余内容。	【截图支撑】截图可清晰看到单条用户信息的返回格式，无多余数据； 【支撑判断】返回结果符合“按 ID 查询单条数据”的业务逻辑，说明正常输入仅能获取对应 ID 的有限数据，支撑“正常查询具有数据范围限制”的判断。
异常输入	页面返回数据库中 users 表的所有用户信息，数据量远超普通输入的单条结果，格式与普通输入一致，但数据范围大幅扩大。	【截图支撑】截图可清晰看到多条用户信息并列显示，与普通输入的单条结果形成鲜明对比； 【支撑判断】该差异说明输入内容篡改了查询条件，让数据库执行了“查询所有用户”的非预期操作，支撑“异常输入可突破正常查询限制”的判断。

4.为什么这种差异不像“正常查询变化”：
正常查询变化的核心特征是“在业务预期范围内，数据量、数据内容随合法输入的不同而合理变化”，例如输入“1”返回 ID=1 的信息，变化仅为“单条数据的切换”，未突破“按 ID 查询单条数据”的业务逻辑，也未改变查询的核心规则； 而本次异常输入与普通输入的差异，完全突破了业务预期和查询规则：一是数据量异常，从“单条数据”变为“所有数据”，远超正常查询的范围；二是查询逻辑异常，输入的特殊字符改变了原有“按 ID 匹配”的查询条件，导致数据库执行非预期操作（查询所有数据、返回空结果）；三是变化无规律，正常查询的变化与输入的 ID 一一对应，而异常输入的返回结果与输入的“ID 数值”无关（如输入“1’ OR ’1’=’1”，返回所有数据，而非 ID=1 的相关数据），因此这种差异不属于正常查询变化，而是注入漏洞导致的非预期行为。

任务 3：机制分析与差异比较（4.5 分）

1.输入如何进入查询结构:
输入：用户在 DVWA 输入框填写内容（数字/特殊字符）
拼接：后端代码无隔离、无过滤，直接将输入以字符串拼接方式嵌入 SQL 查询语句
解析：数据库把输入内容当作 SQL 语法结构解析，而非普通数据
执行：数据库按被篡改的结构执行查询
回显/行为差异：返回数据范围、内容、状态出现非预期变化
核心结论：输入不是作为“参数值”进入查询，而是作为语句结构的一部分进入，这是漏洞形成的关键路径。

2.为什么根因是“结构被污染”而不是“字符本身危险”：
字符本身不危险：单引号’、OR、--只是普通符号/关键字，单独出现不会造成漏洞
真正危险的是位置：当这些字符被拼入 SQL 查询结构内部，才会改变语句逻辑、破坏原有结构
结构被污染 = 数据变成了代码
正常情况：输入 = 数据值

漏洞情况：输入 = 篡改结构的指令
根因总结：不是字符有毒，是输入进入了不该进入的查询结构，导致结构被污染。

3.Low 与 Medium 的差异在哪里:
Low 等级:无任何过滤、无转义、无防护;直接字符串拼接;输入可任意篡改 SQL 结构;回显明显、极易利用
Medium 等级:对单引号'做转义/过滤;对输入做简单处理;依然使用字符串拼接（根因未修复）;注入难度提升，但结构污染的根因仍然存在
核心差异总结:
Low:完全无防护，结构完全开放被污染
Medium:增加了字符过滤，但没有真正分离结构与数据，根因未解决

4.普通 SQL 注入与 Blind SQL Injection 的差异在哪里:
普通注入:页面直接返回查询结果;能看到用户名、ID、数据内容;证据直观、肉眼可见
盲注（Blind SQL）:页面不返回具体数据;只返回两种状态:存在/显示（真）,不存在/不显示（假）;只能通过页面行为差异判断，无法直接看到内容
最大差异:普通注入看数据，盲注看状态;一个能看见结果，一个只能猜结果

5.为什么盲注和普通注入根因相同，但证据形式不同:
1. 根因完全相同:
都是 输入被拼入查询结构 → 结构被污染
都是 数据与代码未分离
2. 证据形式不同原因:
普通注入：页面开启回显，数据库结果直接输出 → 证据是数据内容
盲注：页面关闭回显，只返回真假状态 → 证据是行为差异
总结：根因一样，只是页面展示能力不同，导致证据不一样。

强制判断题：
1.SQL 注入的根因更接近
A.输入里有几个危险字符
B.输入进入了不该进入的查询结构
答：我选择 B，理由：危险字符本身无害，只有当输入被拼入 SQL 结构、改变查询逻辑时才形成漏洞。根因是结构被污染，不是字符危险。

2.Blind SQL Injection 和普通 SQL 注入的最大差异更接近
A.根因不同
B.证据形式不同
答：我选择 B，理由：两者根因完全一样（都是结构被污染），唯一区别是页面是否回显数据，导致证据呈现形式不同。

你最容易犯的错误
错误：把 payload 当根因。
原因：只盯住字符串表面，没有看到“查询结构”这一层。
改法：每次分析都回到一句话：

问题发生在输入内容本身，还是发生在输入被拼进了查询结构里？
答：问题不在输入内容本身，而在输入被拼进了查询结构里，导致结构被污染。

任务 4：SQL 注入防护清单与解释（3.5 分）

请给出一份防护清单，并说明每条措施控制的是哪一层：
1. 参数化查询：
解决问题：彻底拆分 SQL 语句结构与用户输入数据，不再使用字符串拼接构造查询，用户输入只会被当作普通参数值，无法参与 SQL 语法构造，从源头杜绝查询结构被篡改。
控制层级：根因控制
不能替代什么：无法替代输入校验、账号权限管控；即使使用参数化查询，恶意违规业务输入依然需要拦截，不能单独解决业务层面非法访问问题。
2. 输入校验
解决问题对前端输入点进行规则校验，采用白名单机制，限制输入格式、长度、字符范围，拦截单引号、逻辑运算符、注释符等特殊危险字符，减少恶意 payload 的输入暴露面。
控制层级：暴露面控制
不能替代什么：不能替代参数化查询；过滤规则存在绕过风险，仅靠输入过滤无法根治结构污染的底层漏洞。
3. 错误处理
解决问题：关闭数据库原生报错、后端 SQL 执行错误详情，统一使用自定义友好页面提示，避免通过报错泄露表名、字段名、数据库结构等敏感信息，缩小漏洞利用的线索范围。
控制层级：暴露面控制 + 后果缩减
不能替代什么：无法修复 SQL 注入本身的漏洞根因，只是隐藏攻击线索，攻击者仍可通过盲注等方式持续探测。
4. 数据库最小权限
解决问题：限制网站业务数据库账号权限，仅分配查询、只读必要权限，禁止删除、修改、新建、导出等高危操作，即使发生注入漏洞，也能限制攻击者的破坏范围。
控制层级：后果缩减
不能替代什么：不能阻止漏洞触发与数据越权查询，只能降低攻击造成的破坏程度，不属于漏洞根源修复措施。
5. 补充措施：部署 WAF Web 应用防火墙
解决问题：实时拦截常见 SQL 注入特征载荷，实时阻断恶意访问请求，增加外层安全屏障。
控制层级：暴露面控制
不能替代什么：属于外挂式防护，存在规则绕过，无法替代代码层根因修复。

七．收口项：

收口项 1：SQL 注入因果链/防护关系图（3 分）

流程节点	DVWA 场景具体内容	核心根因	对应防护控制点
输入点	虚拟机访问 DVWA，ID 查询输入框为可控输入点，用户可自主构造任意字符	前端输入完全可控，无前置约束	输入校验（白名单过滤、特殊字符拦截）
查询构造	Low 等级后端直接拼接用户输入，组合成	采用不安全字符	参数化查询（预

流程节点	DVWA 场景具体内容	核心根因	对应防护控制点
	完整 SQL 查询语句	串拼接方式	编译，隔离结构与数据）
数据库解析/执行	数据库识别拼接后完整 SQL 语法，恶意符号被解析为语句结构，执行篡改后查询逻辑	数据混入 SQL 结构，造成结构污染	根因修复：强制语句与数据分离
结果差异或行为证据	普通输入仅返回单条数据；注入输入批量返回全部数据 / 页面无回显（盲注），行为明显异常	业务查询边界被突破，执行逻辑失控	错误处理（隐藏报错、掩盖探测线索）
根因判断	未实现结构与数据分离，输入非法进入查询结构，而非单纯字符危险	结构污染是唯一核心根因	多层组合防护，不依赖单一手段
后果约束	漏洞可导致数据泄露、数据篡改等风险	权限过大会放大攻击危害	数据库最小权限配置

收口项 2：学生自评表（2 分）

自评问题	具体回答
1. 我本章最清楚的一点是什么？	最清楚的一点是 SQL 注入的核心根因是“查询结构被污染”，而非输入字符本身危险。通过 DVWA Low 等级的实验的现象，能清晰理解“输入被拼入查询结构”如何篡改 SQL 逻辑，也能区分根因控制（参数化查询）与后果缩减（最小权限）的不同作用。
2. 我最容易混淆的一点是什么？	最容易混淆的是普通 SQL 注入与盲注的差异，初期容易误以为两者根因不同，经过分析后才明确，两者根因完全一致，仅证据形式不同（普通注入看数据回显，盲注看页面行为状态），容易把“证据形式差异”误判为“根因差异”。
3. 我本次作业中哪一段最需要教师复核？	最需要教师复核的是任务 4 中“防护措施的层级区分”部分，尤其是错误处理的“暴露面控制+后果缩减”双重层级判断，不确定是否准确区分了暴露面与后果缩减的边界，希望教师指导确认分层逻辑是否正确。
4. 如果重做一次，我会优先改哪一部分？	优先修改任务 2 中“返回结果差异的解释”部分，本次作业虽补充了截图支撑和判断，但对“异常输入与正常输入差异的本质区别”表述不够精炼，重做时会更聚焦“业务逻辑突破”这一核心，让差异解释更贴合根因，避免冗余表述。

收口项 3：AI 输出审核与人工复核记录（必做）

1.本作业哪些内容使用过 AI 辅助：

答：辅助解释部分专业术语的应用场景（如“结构污染”“暴露面控制”），结合 DVWA 场景补充具体表述，避免概念抄写。

2.哪些判断是你自己复核后保留的：

答：1.所有实验场景均限定在“localhostDVWA 授权环境”，未涉及公网目标、未授权系统，符合授权边界要求，该判断经人工复核后保留； 2.SQL 注入根因判断为“结构被污染”，而非“字符本身危险”，强制判断题的选择及理由，经结合 DVWA 实验现象人工复核，判断准确，予以保留； 3.防护措施的层级区分（参数化查询为根因控制、最小权限为后果缩减），经对照分层逻辑人工复核，符合要求，予以保留； 4.因果链关系表的节点对应关系，经核对“输入→拼接→解析→执行→差异”主线，逻辑完整，予以保留。

3.哪些内容你发现 AI 输出不准确并进行了修正：

答：初始 AI 输出中，任务 3“Low 与 Medium 差异”部分，未明确“Medium 等级依然存在结构污染根因”，仅表述为“增加过滤”，人工修正后补充“Medium 等级依然使用字符串拼接，根因未修复”，确保差异分析贴合根因核心；

4.你最终如何确认结论和证据是一一对应的：

答：1.逐句核对各任务的结论与实验证据：如“异常输入可篡改查询逻辑”的结论，对应任务 2 中“输入“1’ OR ’1’=’1”返回所有用户数据”的现象，确保每个结论都有具体的 DVWA 实验现象（输入、回显差异）支撑； 2.核对防护措施与漏洞节点的对应关系：如“参数化查询”对应“查询构造环节的字符串拼接漏洞”，确保防护措施能精准解决对应环节的问题；3.人工复核所有逻辑链条，确保“输入→拼接→解析→执行→差异”的因果关系连贯，结论（根因、漏洞存在）与证据（现象、回显）无矛盾，最终确认所有结论均有对应的实验证据支撑，无无证据结论或证据与结论不匹配的情况。